

Chapter 27

HMAC Accelerator (HMAC)

The Hash-based Message Authentication Code (HMAC) module computes Message Authentication Codes (MACs) using hash algorithm SHA-256 and keys as described in RFC 2104. The 256-bit HMAC key is stored in an eFuse key block and can be set as read-protected, i.e., the key is not accessible from outside the HMAC accelerator.

27.1 Main Features

- Standard HMAC-SHA-256 algorithm
- HMAC-SHA-256 calculation based on key in eFuse,
 - whose result cannot be accessed by software in downstream mode for high security
 - whose result can be accessed by software in upstream mode
- Generates required keys for the RSA Digital Signature Peripheral (RSA_DS) in downstream mode
- Re-enables soft-disabled JTAG in downstream mode

27.2 Functional Description

The HMAC module operates in two modes: upstream mode and downstream mode. In upstream mode, users provide the HMAC message and read back the calculation result. In downstream mode, the HMAC module provides input to two possible other internal hardware modules: On the one hand, an HMAC can be used to enable JTAG after JTAG has been temporarily disabled before. On the other hand, an HMAC can be used as decryption key for Digital Signature parameters stored in flash memory for the RSA_DS peripheral. Furthermore, the calculations happen internally and automatically in downstream mode, so that confidentiality of any key and derived key material is ensured, given correct configuration.

Note:

After the reset signal being released, the HMAC module will check whether the RSA_DS_KEY exists in the eFuse. If the key exists, the HMAC module will enter downstream RSA_DS mode and finish the RSA_DS_KEY calculation automatically. This process is automatically completed by hardware and does not require software participation. When the downstream operation after reset is completed, the HMAC will automatically return to the idle state.

27.2.1 Upstream Mode

To calculate the HMAC value in upstream mode, users should perform the following steps:

1. Initialize the HMAC module and enter upstream mode.

2. Write the correctly padded message to the HMAC, one block at a time.
3. Read back the result from HMAC.

For details of this process, please see Section [27.2.5](#).

Note:

Common use cases for the upstream mode are challenge-response protocols supporting HMAC-SHA-256. Assume the two entities in the challenge-response protocol are A and B respectively, and the data message they expect to exchange is M. The general authentication process of this protocol is as follows:

- A calculates a unique random number M.
- A sends M to B.
- B calculates the HMAC (through M and KEY) and sends the result to A.
- A calculates the HMAC (through M and KEY) internally.
- A compares the two results. If the results are the same, then the identity of B is authenticated.

27.2.2 Downstream Mode - JTAG Enable Feature

JTAG debugging can be disabled by eFuse in a way which allows later re-enabling using the HMAC module. (For more details, please see Chapter [8 eFuse Controller \(EFUSE\)](#).) The HMAC module will expect the user to supply the HMAC result for one of the eFuse keys. The HMAC module will check whether the supplied HMAC matches the one calculated from the chosen key. If both HMACs are the same, JTAG will be enabled until the user calls the HMAC module to clear the results and consequently disable JTAG again.

To re-enable JTAG, users should perform the following steps:

1. Enable the HMAC module by initializing clock and reset signals of HMAC, and enter downstream JTAG enable mode by configuring [HMAC_SET_PARA_PURPOSE_REG](#). Then, wait for the calculation to complete. Please see Section [27.2.5](#) for more details.
2. Write 1 to the [HMAC_SOFT_JTAG_CTRL_REG](#) register to enter JTAG re-enable mode.
3. Write the 256-bit HMAC value to register [HMAC_WR_JTAG_REG](#). This value is obtained by performing a local HMAC calculation from the 32-byte 0x00 using SHA-256 and the key that has been written to the eFuse. It needs to be written 8 times and 32-bit each time in big-endian word order.
4. If the HMAC result calculated from the key in the eFuse matches the value that users wrote in step 3, then JTAG is re-enabled. Otherwise, JTAG remains disabled.
5. After writing 1 to [HMAC_SET_INVALIDATE_JTAG_REG](#) or resetting the chip, JTAG will be disabled. If users want to re-enable JTAG again, they need to repeat the above steps again.

27.2.3 Downstream Mode - RSA_DS Key Derivation Feature

The RSA Digital Signature Peripheral (RSA_DS) encrypts its parameters using the AES-CBC algorithm. The HMAC module is used as a Key Derivation Function (KDF) to derive the AES key to decrypt these parameters (parameter decryption key).

Before starting the RSA_DS peripheral, users need to obtain the parameter decryption key for the RSA_DS peripheral through HMAC calculation. For more information, please see Chapter [30 RSA Digital Signature](#)

Peripheral (RSA_DS). After the chip is powered on, the HMAC module will check whether the key required to calculate the parameter decryption key has been burned in the eFuse block. If the key has been burned, HMAC module will automatically enter the downstream RSA_DS algorithm mode and complete the HMAC calculation based on the chosen key.

27.2.4 HMAC eFuse Configuration

Each HMAC key burned into an eFuse block has a key purpose, specifying for which functionality the key can be used. The HMAC module will not accept a key with a non-matching purpose for any functionality. The HMAC module provides three different functionalities: re-enabling JTAG, RSA_DS KDF in downstream mode, and pure HMAC calculation in upstream mode. For each functionality, there exists a corresponding key purpose, listed in Table 27.2-1. Additionally, another purpose specifies a key which may be used for re-enabling JTAG as well as for serving as RSA_DS KDF.

Before enabling HMAC to do calculations, user should make sure the key to be used has been burned in eFuse by reading the registers EFUSE_KEY_PURPOSE_x (We have a total of 6 keys in eFuse, so the value of x is 0 ~ 5. Among which, EFUSE_KEY_PURPOSE_0 ~ EFUSE_KEY_PURPOSE_1 belong to the register EFUSE_RD_REPEAT_DATA1_REG, and EFUSE_KEY_PURPOSE_2 ~ EFUSE_KEY_PURPOSE_5 belong to the register EFUSE_RD_REPEAT_DATA2_REG) from *8 eFuse Controller (EFUSE)*. Take upstream mode as an example, if there is no EFUSE_KEY_PURPOSE_HMAC_UP in EFUSE_KEY_PURPOSE_0 ~ 5, it means there is no key in eFuse that can be used for the HMAC upstream mode. Users can burn a key to eFuse as follows:

1. Prepare a secret 256-bit HMAC key and burn the key to an empty eFuse block y . As there are 6 blocks for storing a key in eFuse and the numbers of those blocks range from 4 to 9, the value of y is 4 ~ 9. Hence, when talking about key0, it means eFuse block4. Then, program the purpose to EFUSE_KEY_PURPOSE_ $(y - 4)$. Take upstream mode as an example: after programming the key, the user should program EFUSE_KEY_PURPOSE_HMAC_UP (corresponding value is 8) to EFUSE_KEY_PURPOSE_ $(y - 4)$. Please see Chapter *8 eFuse Controller (EFUSE)* on how to program eFuse keys.
2. If needed, configure this eFuse key block to be read protected, so that users cannot read its value. A copy of this key should be kept by any party who needs to verify this device.

Please note that the key whose purpose is EFUSE_KEY_PURPOSE_HMAC_DOWN_ALL can be used for both re-enabling JTAG or RSA_DS.

Table 27.2-1. HMAC Purposes and Configuration Value

Purpose	Mode	Value	Description
JTAG Re-enable	Downstream	6	EFUSE_KEY_PURPOSE_HMAC_DOWN_JTAG
RSA_DS KDF	Downstream	7	EFUSE_KEY_PURPOSE_HMAC_DOWN_DIGITAL_SIGNATURE
HMAC Calculation	Upstream	8	EFUSE_KEY_PURPOSE_HMAC_UP
Both JTAG Re-enable and RSA_DS KDF	Downstream	5	EFUSE_KEY_PURPOSE_HMAC_DOWN_ALL

Select eFuse Key Blocks and HMAC Purposes

The eFuse controller provides six key blocks, i.e., KEY0 ~ 5. To select a particular KEY n for an HMAC calculation, write the key number n to register HMAC_SET_PARA_KEY_REG.

Write a correct purpose to register [HMAC_SET_PARA_PURPOSE_REG](#) (see Section 27.2.5). Note that the purpose of the key has also been programmed to eFuse memory. Only when the configured HMAC purpose matches the defined purpose of KEY n , the HMAC module will execute the configured calculation. Otherwise, it will return a matching error and stop the current calculation.

For example, suppose a user selects KEY3 for HMAC calculation, and the value programmed to [EFUSE_KEY_PURPOSE_3](#) is 6 (EFUSE_KEY_PURPOSE_HMAC_DOWN_JTAG). Based on Table 27.2-1, KEY3 can be used to re-enable JTAG. If the value written to register [HMAC_SET_PARA_PURPOSE_REG](#) is also 6, then the HMAC module will start the process to re-enable JTAG.

Select Key from Key Manager

Users can deploy the HMAC key in the Key Manager. For deployment procedures, please refer to Section 34 [Key Manager](#). To select this key from the Key Manager, write the key index 7 to the register [HMAC_SET_PARA_KEY_REG](#). When a key is selected from the Key Manager, the intended operation target in the register is automatically set to "HMAC Calculation", and all other configurations will be ignored.

27.2.5 HMAC Process (Detailed)

The process for users to call HMAC in ESP32-P4 is as follows:

27.2.5.1 Enable HMAC Module

1. Set the peripheral clock bits for HMAC and SHA peripherals in register [HP_SYS_CLKRST_REG_CRYPTO_HMAC_CLK_EN](#), and clear the corresponding peripheral reset bits in register [HP_SYS_CLKRST_REG_RST_EN_HMAC](#). For information on those registers, please see Chapter 10 [Reset and Clock](#).
2. Write 1 to register [HMAC_SET_START_REG](#).

27.2.5.2 Configure HMAC Keys and Key Purposes

1. Write the key purpose m to register [HMAC_SET_PARA_PURPOSE_REG](#). The possible key purpose values are shown in Table 27.2-1. For more information, please refer to Section 27.2.4.
2. Select KEY n in eFuse memory as the key by writing n (ranges from 0 to 5, and 7) to register [HMAC_SET_PARA_KEY_REG](#). For more information, please refer to Section 27.2.4.
3. Write 1 to register [HMAC_SET_PARA_FINISH_REG](#) to complete the configuration.
4. Read register [HMAC_QUERY_ERROR_REG](#). If its value is 1, it means the purpose of the selected block does not match the configured key purpose and the calculation will not proceed. If its value is 0, it means the purpose of the selected block matches the configured key purpose, and then the calculation can proceed.
5. When the value of [HMAC_SET_PARA_PURPOSE_REG](#) is not 8, it means the HMAC module is in downstream mode, proceed with 27.2.5.3. When the value is 8, it means the HMAC module is in upstream mode, proceed with 27.2.5.4.

27.2.5.3 Downstream Mode Process

1. Poll Status register [HMAC_QUERY_BUSY_REG](#) until it reads 0.

2. To clear the result and make further usage of the dependent hardware (JTAG or RSA_DS) impossible, write 1 to either register [HMAC_SET_INVALIDATE_JTAG_REG](#) to clear the result generated by the JTAG key; or to register [HMAC_SET_INVALIDATE_DS_REG](#) to clear the result generated by RSA_DS_KEY. Afterwards, the HMAC process needs to be restarted to re-enable any of the dependent peripherals.

27.2.5.4 Upstream Mode Process

1. Write message in upstream mode:
 - (a) Poll Status register [HMAC_QUERY_BUSY_REG](#) until it reads 0.
 - (b) Apply SHA padding to message as described in Section 27.3.1, and get message block: Block_*n* (*n* ≥ 0)
 - (c) Perform one of the following operations according to the block number *n*:
 - If there is only one message block in total which has included all padding bits:
 - i. Write the 512-bit Block_*n* to register [HMAC_WR_MESSAGE_n_REG](#) (*n*: 0-15). Write 1 to register [HMAC_SET_MESSAGE_ONE_REG](#), to trigger the processing of this message block.
 - ii. Poll Status register [HMAC_QUERY_BUSY_REG](#) until it reads 0.
 - iii. Write 1 to register [HMAC_ONE_BLOCK_REG](#), and finish message transmission.
 - If Block_*n* is the last padded block:
 - i. Write the 512-bit Block_0 to register [HMAC_WR_MESSAGE_n_REG](#) (*n*: 0-15). Write 1 to register [HMAC_SET_MESSAGE_ONE_REG](#), to trigger the processing of this message block.
 - ii. Poll Status register [HMAC_QUERY_BUSY_REG](#) until it reads 0.
 - iii. Message transmission finished automatically.
 - If Block_*n* is the second last padded block:
 - i. Write the 512-bit Block_*n* to register [HMAC_WR_MESSAGE_n_REG](#) (*n*: 0-15). Write 1 to register [HMAC_SET_MESSAGE_ONE_REG](#), to trigger the processing of this message block.
 - ii. Poll Status register [HMAC_QUERY_BUSY_REG](#) until it reads 0.
 - iii. Write 1 to register [HMAC_SET_MESSAGE_PAD_REG](#), and continue message transmission (jump back to step (c)).
 - If Block_*n* is neither the last nor the second last message block:
 - i. Write the 512-bit Block_*n* to register [HMAC_WR_MESSAGE_n_REG](#) (*n*: 0-15). Write 1 to register [HMAC_SET_MESSAGE_ONE_REG](#), to trigger the processing of this message block.
 - ii. Poll Status register [HMAC_QUERY_BUSY_REG](#) until it reads 0.
 - iii. Write 1 to register [HMAC_SET_MESSAGE_ING_REG](#), and continue message transmission (jump back to step (c)).
2. Read hash result in upstream mode:
 - (a) Poll Status register [HMAC_QUERY_BUSY_REG](#) until it reads 0.
 - (b) Read hash result from register [HMAC_RD_RESULT_n_REG](#) (*n*: 0-7).

- (c) Write 1 to register [HMAC_SET_RESULT_FINISH_REG](#) to finish calculation. The result will be cleared at the same time.
- (d) Upstream mode operation is completed.

Note:

The SHA accelerator can be called directly, or used internally by the RSA_DS peripheral and the HMAC module. However, they can not share the hardware resources simultaneously. Therefore, the SHA module must not be called neither by the CPU nor by the RSA_DS peripheral when the HMAC module is in use.

27.3 HMAC Algorithm Details

27.3.1 Padding Bits

The HMAC module uses SHA-256 as hash algorithm. If the input message is not a multiple of 512 bits, the user must apply a SHA-256 padding algorithm in software. The SHA-256 padding algorithm is the same as described in Section *Padding the Message* of [FIPS PUB 180-4](#). In downstream mode, users do not need to input any message or apply padding. The HMAC module uses a default 32-byte pattern of 0x00 for re-enabling JTAG and a 32-byte pattern of 0xff for deriving the AES key for the RSA_DS peripheral.

For the convenience of reading, here we will briefly describe the process of message padding. As shown in Figure 27.3-1, suppose the length of the unpadded message is m bits. Padding steps are as follows:

1. Append one bit of value "1" to the end of the unpadded message.
2. Append k bits of value "0", where k is the smallest non-negative number which satisfies $m + 1 + k \equiv 448 \pmod{512}$.
3. Append a 64-bit integer value as a binary block. This block consists of the length of the unpadded message as a big-endian binary integer value m .

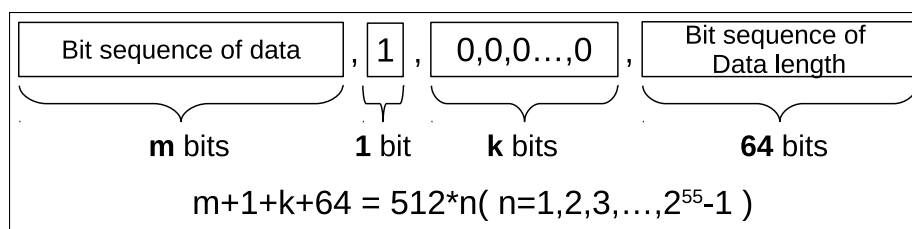


Figure 27.3-1. HMAC SHA-256 Padding Diagram

In upstream mode, if the length of the unpadded message is a multiple of 512 bits, users can configure hardware to apply SHA padding by writing 1 to [HMAC_SET_MESSAGE_END_REG](#) or do padding work themselves by writing 1 to [HMAC_SET_MESSAGE_PAD_REG](#). If the length is not a multiple of 512 bits, SHA padding must be manually applied by the user. After the user prepared the padding data, they should complete the subsequent configuration according to the Section 27.2.5.

In Figure [27.3-2](#):

1. ipad is a 512-bit message block composed of 64 bytes of 0x36.
2. opad is a 512-bit message block composed of 64 bytes of 0x5c.

The HMAC module appends a 256-bit 0 sequence after the bit sequence of the 256-bit key K in order to get a 512-bit K_0 . Then, the HMAC module XORs K_0 with ipad to get the 512-bit S1. Afterwards, the HMAC module appends the input message (multiple of 512 bits) after the 512-bit S1, and exercises the SHA-256 algorithm to get the 256-bit H1.

The HMAC module appends the 256-bit SHA-256 hash result H1 to the 512-bit S2 value, which is calculated using the XOR operation of K_0 and opad. A 768-bit sequence will be generated. Then, the HMAC module uses the SHA padding algorithm described in Section [27.3.1](#) to pad the 768-bit sequence to a 1024-bit sequence, and applies the SHA-256 algorithm to get the final hash result (256-bit).

27.4 Register Summary

The addresses in this section are relative to HMAC Accelerator base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section [Access Types for Registers](#).

Name	Description	Address	Access
Control/Status Registers			
HMAC_SET_START_REG	HMAC start control register	0x0040	WO
HMAC_SET_PARA_FINISH_REG	HMAC configuration completion register	0x004C	WO
HMAC_SET_MESSAGE_ONE_REG	HMAC message control register	0x0050	WO
HMAC_SET_MESSAGE_ING_REG	HMAC message continue register	0x0054	WO
HMAC_SET_MESSAGE_END_REG	HMAC message end register	0x0058	WO
HMAC_SET_RESULT_FINISH_REG	HMAC result reading finish register	0x005C	WO
HMAC_SET_INVALIDATE_JTAG_REG	Invalidate JTAG result register	0x0060	WO
HMAC_SET_INVALIDATE_DS_REG	Invalidate RSA_DS result register	0x0064	WO
HMAC_QUERY_ERROR_REG	Stores matching results between keys generated by users and corresponding purposes	0x0068	RO
HMAC_QUERY_BUSY_REG	Busy state of HMAC module	0x006C	RO
HMAC_SET_MESSAGE_PAD_REG	Software padding register	0x00F0	WO
HMAC_ONE_BLOCK_REG	One block message register	0x00F4	WO
Configuration Registers			
HMAC_SET_PARA_PURPOSE_REG	HMAC parameter configuration register	0x0044	WO
HMAC_SET_PARA_KEY_REG	HMAC parameters configuration register	0x0048	WO
HMAC_SOFT_JTAG_CTRL_REG	Re-enable JTAG register 0	0x00F8	WO
HMAC_WR_JTAG_REG	Re-enable JTAG register 1	0x00FC	WO
HMAC Message Block			
HMAC_WR_MESSAGE_0_REG	Message register 0	0x0080	WO
HMAC_WR_MESSAGE_1_REG	Message register 1	0x0084	WO
HMAC_WR_MESSAGE_2_REG	Message register 2	0x0088	WO
HMAC_WR_MESSAGE_3_REG	Message register 3	0x008C	WO
HMAC_WR_MESSAGE_4_REG	Message register 4	0x0090	WO
HMAC_WR_MESSAGE_5_REG	Message register 5	0x0094	WO
HMAC_WR_MESSAGE_6_REG	Message register 6	0x0098	WO
HMAC_WR_MESSAGE_7_REG	Message register 7	0x009C	WO
HMAC_WR_MESSAGE_8_REG	Message register 8	0x00A0	WO
HMAC_WR_MESSAGE_9_REG	Message register 9	0x00A4	WO
HMAC_WR_MESSAGE_10_REG	Message register 10	0x00A8	WO
HMAC_WR_MESSAGE_11_REG	Message register 11	0x00AC	WO
HMAC_WR_MESSAGE_12_REG	Message register 12	0x00B0	WO
HMAC_WR_MESSAGE_13_REG	Message register 13	0x00B4	WO
HMAC_WR_MESSAGE_14_REG	Message register 14	0x00B8	WO
HMAC_WR_MESSAGE_15_REG	Message register 15	0x00BC	WO
HMAC Upstream Result			

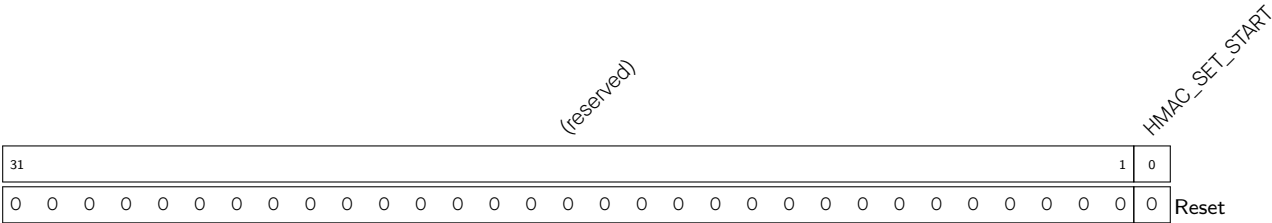
Name	Description	Address	Access
HMAC_RD_RESULT_0_REG	Hash result register 0	0x00C0	RO
HMAC_RD_RESULT_1_REG	Hash result register 1	0x00C4	RO
HMAC_RD_RESULT_2_REG	Hash result register 2	0x00C8	RO
HMAC_RD_RESULT_3_REG	Hash result register 3	0x00CC	RO
HMAC_RD_RESULT_4_REG	Hash result register 4	0x00D0	RO
HMAC_RD_RESULT_5_REG	Hash result register 5	0x00D4	RO
HMAC_RD_RESULT_6_REG	Hash result register 6	0x00D8	RO
HMAC_RD_RESULT_7_REG	Hash result register 7	0x00DC	RO
Version Register			
HMAC_DATE_REG	Version control register	0x01FC	R/W

27.5 Registers

The addresses in this section are relative to HMAC Accelerator base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

For how to program reserved fields, please refer to Section [Programming Reserved Register Field](#).

Register 27.1. HMAC_SET_START_REG (0x0040)

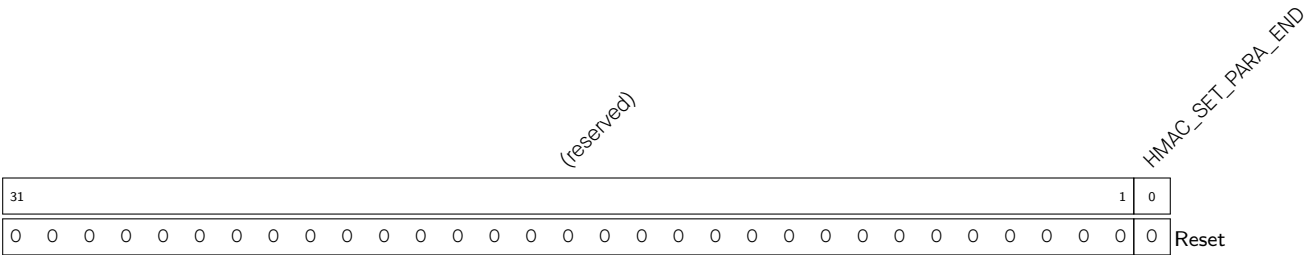


HMAC_SET_START Configures whether or not to enable HMAC.

- 0: Disable HMAC
- 1: Enable HMAC

(WO)

Register 27.2. HMAC_SET_PARA_FINISH_REG (0x004C)

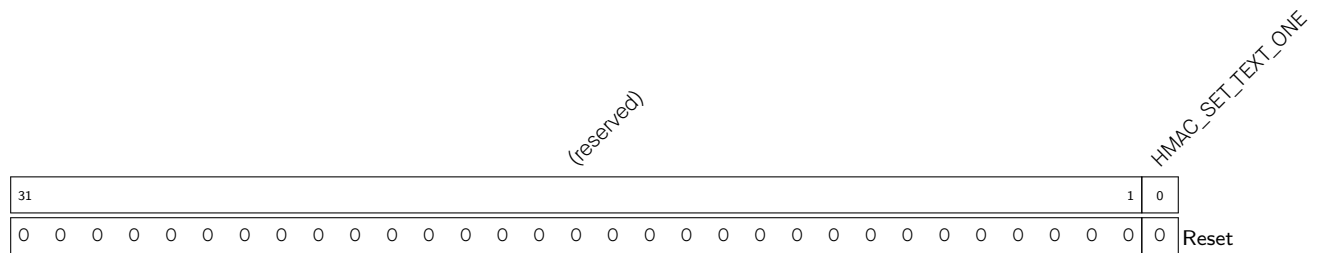


HMAC_SET_PARA_END Configures whether to finish HMAC configuration.

- 0: No effect
- 1: Finish configuration

(WO)

Register 27.3. HMAC_SET_MESSAGE_ONE_REG (0x0050)



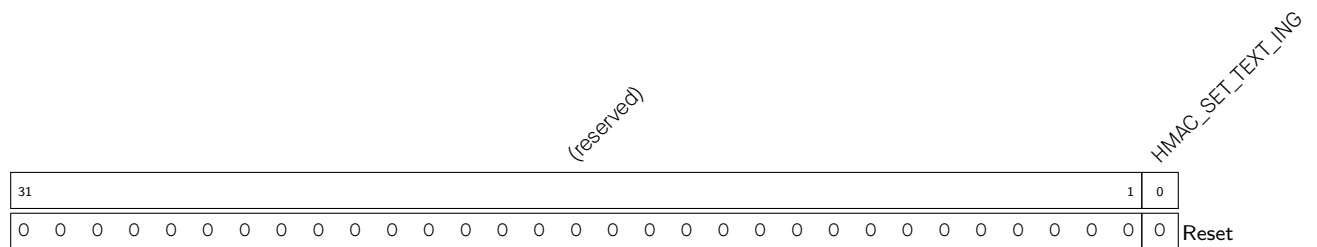
HMAC_SET_TEXT_ONE Calls SHA to calculate one message block.

0: No effect

1: Calls SHA to calculate one message block.

(WO)

Register 27.4. HMAC_SET_MESSAGE_ING_REG (0x0054)



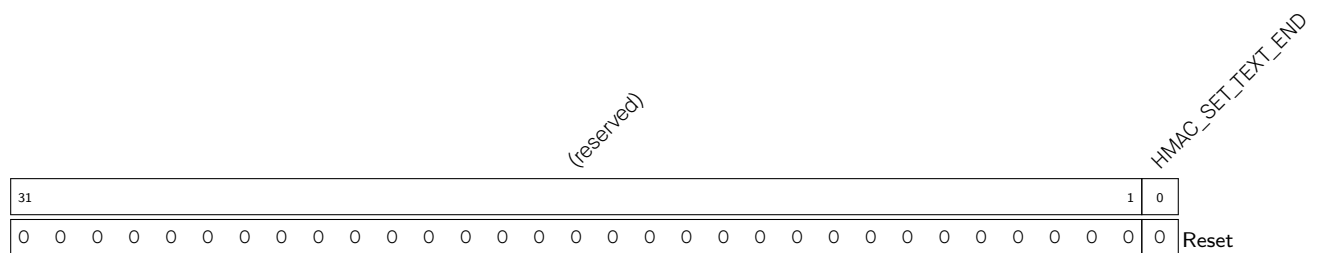
HMAC_SET_TEXT_ING Configures whether or not there are unprocessed message blocks.

0: No unprocessed message block

1: There are still some message blocks to be processed.

(WO)

Register 27.5. HMAC_SET_MESSAGE_END_REG (0x0058)



HMAC_SET_TEXT_END Configures whether to start hardware padding.

0: No effect

1: Start hardware padding

(WO)

(reserved)

HMAC_SET_RESULT_END

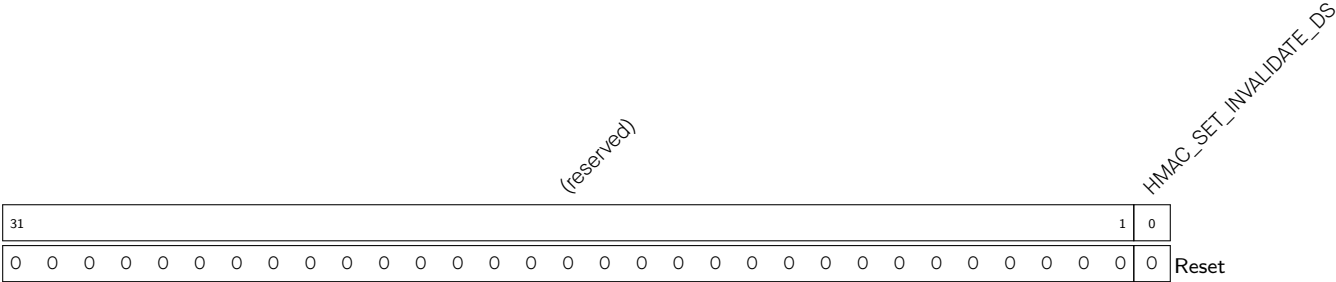
0: Not exit
1: Exit upstream mode and clear calculation results.
(WO)

(reserved)

HMAC_SET_INVALIDATE_JTAG

0: Not clear
1: Clear calculation results
(WO)

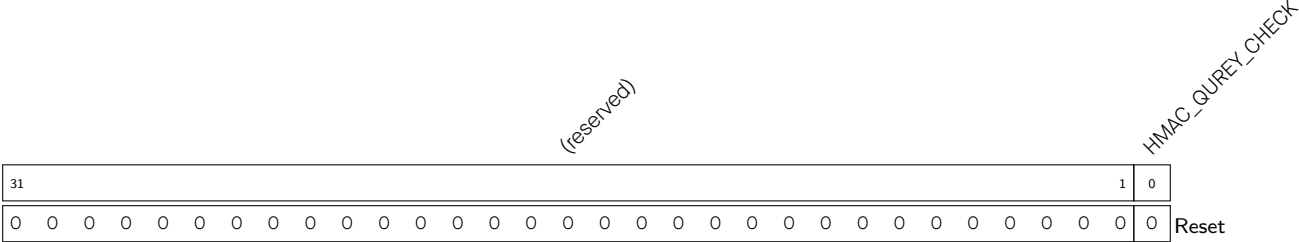
Register 27.8. HMAC_SET_INVALIDATE_DS_REG (0x0064)



HMAC_SET_INVALIDATE_DS Configures whether or not to clear calculation results of the RSA_DS peripheral in downstream mode.

- 0: Not clear
- 1: Clear calculation results (WO)

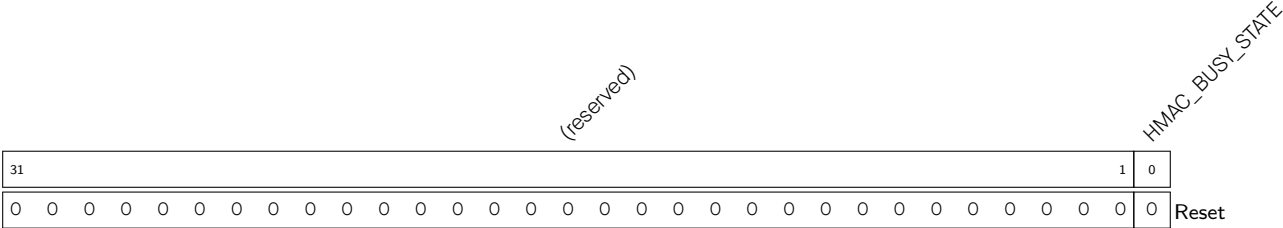
Register 27.9. HMAC_QUERY_ERROR_REG (0x0068)



HMAC_QUIREY_CHECK Represents whether or not an HMAC key matches the purpose.

- 0: Match
- 1: Error (RO)

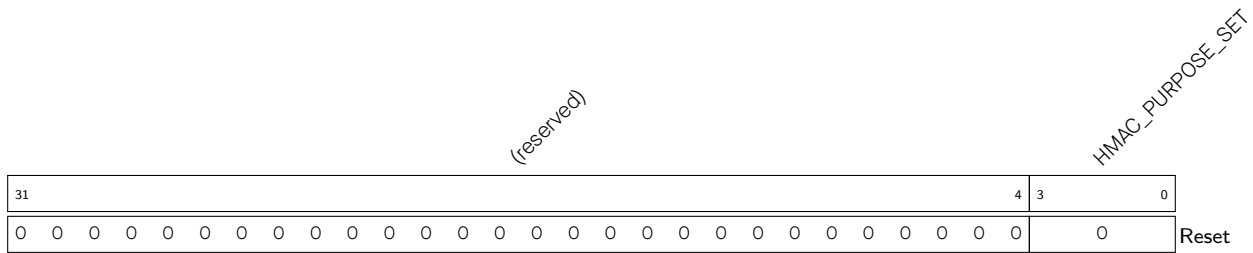
Register 27.10. HMAC_QUERY_BUSY_REG (0x006C)



HMAC_BUSY_STATE Represents whether or not HMAC is in a busy state. Before configuring HMAC, please make sure HMAC is in an IDLE state.

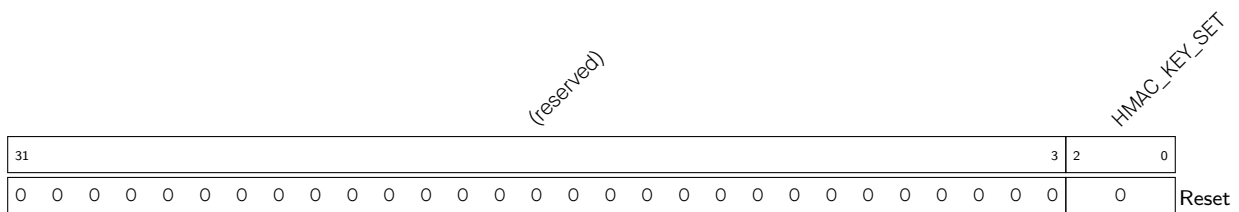
- 0: Idle
- 1: HMAC is still working on the calculation (RO)

Register 27.11. HMAC_SET_PARA_PURPOSE_REG (0x0044)

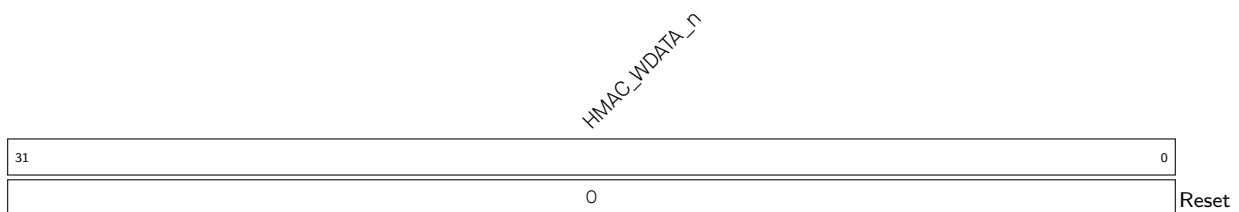


HMAC_PURPOSE_SET Configures the HMAC purpose, refer to the Table 27.2-1. (WO)

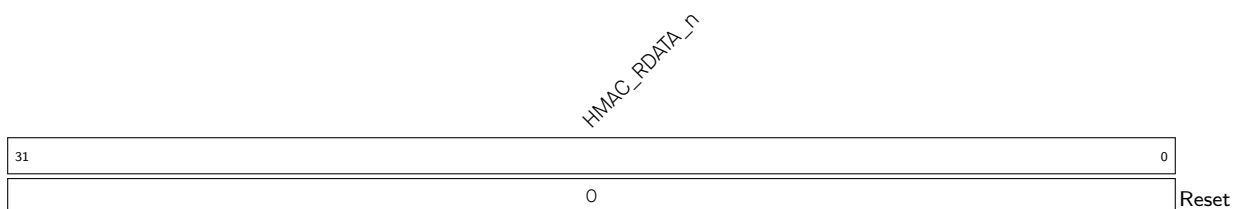
Register 27.12. HMAC_SET_PARA_KEY_REG (0x0048)



HMAC_KEY_SET Configures HMAC key. There are seven keys with index 0 ~ 5, and 7. Write the index of the selected key to this field. (WO)

Register 27.13. HMAC_WR_MESSAGE_n_REG (*n*: 0-15) (0x0080+4**n*)

HMAC_WDATA_n Represents the *n*th 32-bit of message. (WO)

Register 27.14. HMAC_RD_RESULT_n_REG (*n*: 0-7) (0x00C0+4**n*)

HMAC_RDATA_n Represents the *n*th 32-bit of hash result. (RO)

Register 27:15. HMAC_SET_MESSAGE_PAD_REG (0x00F0)

(reserved)																															HMAC_SET_TEXT_PAD																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		
31																															1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

HMAC_SET_TEXT_PAD Configures whether or not the padding is applied by software.

0: Not applied by software

1: Applied by software

(WO)

Register 27:16. HMAC_ONE_BLOCK_REG (0x00F4)

(reserved)																														HMAC_SET_ONE_BLOCK	
31																														1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

HMAC_SET_ONE_BLOCK Write 1 to indicate there is only one block which already contains padding bits and there is no need for padding. (WO)

Register 27:17. HMAC_SOFT_JTAG_CTRL_REG (0x00F8)

(reserved)																														HMAC_SOFT_JTAG_CTRL	
31																														1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

HMAC_SOFT_JTAG_CTRL Configures whether or not to enable JTAG authentication mode.

0: Disable

1: Enable

(WO)

Register 27.18. HMAC_WR_JTAG_REG (0x00FC)

		HMAC_WR_JTAG	
31			0
		x	Reset

HMAC_WR_JTAG Writes the comparing input used for re-enabling JTAG. (WO)

Register 27.19. HMAC_DATE_REG (0x01FC)

		HMAC_DATE	
(reserved)			
31	30	29	0
0	0	0x20200618	Reset

HMAC_DATE Version control register. (R/W)